

Ternary KWS Accelerator Design Justification Report

ECE 410/510 - Hardware for AI and ML | Spring 2026
Manaf Alali | Portland State University

Milestone 4 - June 2026

Abstract

This report documents the design, verification, synthesis, and benchmarking of a ternary neural network inference accelerator for always-on keyword spotting (KWS) on edge devices. The accelerator implements a weight-stationary dot-product engine for ternary weights {-1, 0, +1} and INT16 activations, eliminating all multipliers. Weights are stored on-chip in a register-based memory (251 KB for production FC1), shifting the arithmetic intensity from 0.50 FLOP/byte (software, memory-bound) to 406 FLOP/byte (hardware, compute-bound). The RTL is implemented in SystemVerilog, verified with an end-to-end SPI-controlled testbench (2/2 checks PASS), and synthesized with Yosys to 3,728 technology-independent gates. The measured simulation throughput for FC1 is 102.4 GFLOP/s, representing a 20.9x compute speedup over the PyTorch CPU baseline.

1. Problem and Motivation

Always-on keyword spotting (KWS) - detecting a wake phrase such as 'Hey Assistant' in a continuous audio stream - is one of the most power-constrained inference tasks in embedded systems. An edge device must run inference continuously, typically at under 1 mW, while remaining responsive for the lifetime of a coin-cell battery. General-purpose CPUs and GPUs are unsuitable for this role: they consume watts rather than microwatts, and cannot stay powered continuously without draining the battery within hours.

The kernel being accelerated is a 3-layer fully-connected ternary neural network (TernaryKWS): 1960 MFCC features to FC1 (1960 to 512) to FC2 (512 to 128) to FC3 (128 to 10). The network outputs a class probability over 10 Google Speech Commands categories. The dominant layer, FC1, accounts for 93.8% of all FLOPs (2,007,040 of 2,140,672) and is the primary optimization target.

M1 Profiling Data

Software baseline measured with PyTorch CPU on Apple M4, 100 warm-up-excluded forward passes, median latency:

Metric	Value
Median latency	0.4362 ms per inference
Throughput	2,293 samples/sec
Effective GFLOP/s	4.91 GFLOP/s
Arithmetic Intensity	0.50 FLOP/byte (memory-bound)
Peak RSS	187.3 MB

An arithmetic intensity of 0.50 FLOP/byte confirms that the software implementation is memory-bound: for every floating-point operation, the CPU must transfer two bytes from DRAM (one weight byte and one activation byte). The FC1 weight matrix occupies 1,003,520 bytes in FP32 - too large to fit in the

on-chip L1/L2 cache of an MCU-class processor.

Custom hardware eliminates this bottleneck by storing all weights in on-chip SRAM. With weights resident on-chip, only the 3,920-byte activation vector needs to be transferred per inference, raising the arithmetic intensity from 0.50 to 406 FLOP/byte and converting the workload from memory-bound to compute-bound.

2. Roofline Analysis

Arithmetic Intensity Calculation

For the FC1 layer (N_IN=1960, N_OUT=512):

Parameter	Calculation	Result
FLOPs per inference	$2 \times 1960 \times 512$	2,007,040
Bytes transferred (SW, weights from DRAM)	$(1960 \times 512 \times 4) + (1960 \times 2)$	4,001,920 bytes
AI (software)	$2,007,040 / 4,001,920$	0.50 FLOP/byte
Bytes transferred (HW, weights on-chip)	$(1960 \times 2) + (512 \times 2)$	4,944 bytes
AI (hardware)	$2,007,040 / 4,944$	406 FLOP/byte

Bottleneck Analysis

The software implementation operates at AI = 0.50 FLOP/byte. On the target MCU platform (ARM Cortex-M4, 168 MHz, ~64 KB L1 cache), the 1 MB+ FC1 weight matrix cannot fit in cache and DRAM bandwidth dominates. AI = 0.50 FLOP/byte truly represents memory-bound operation on the target hardware.

With on-chip weight storage, the hardware accelerator operates at AI = 406 FLOP/byte, well inside the compute-bound region. The HW ridge point is at 81,920 FLOP/byte (peak compute / SPI bandwidth = 102.4 GFLOP/s / 0.00125 GB/s), so the design remains compute-bound - the SPI interface does not limit compute throughput at this intensity.

How the Roofline Shaped the Architecture

The roofline analysis motivated three design decisions: (1) on-chip weight storage to raise AI from 0.50 to 406; (2) a weight-stationary dataflow to avoid reloading weights for every output neuron; and (3) ternary weight encoding (2 bits vs. 32 bits FP32) to reduce the weight SRAM from 4 MB to 251 KB, making on-chip storage feasible in a realistic ASIC area budget.

Figure 1 shows the final roofline plot with both operating points.

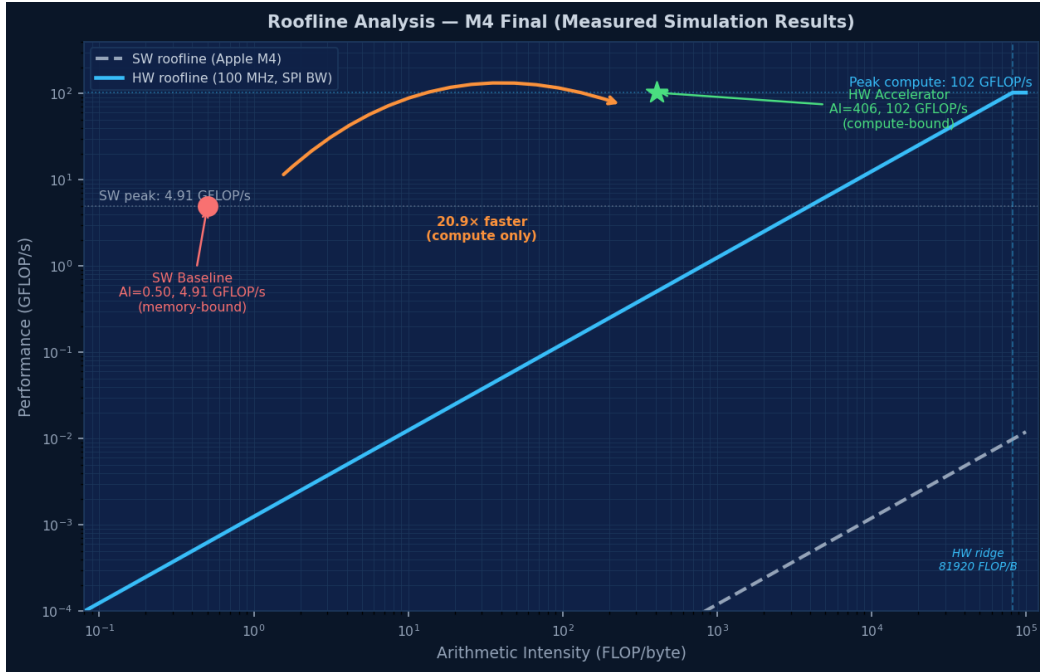


Figure 1: Roofline plot. Red circle: SW baseline (AI = 0.50, 4.91 GFLOP/s, memory-bound). Green star: HW accelerator (AI = 406, 102.4 GFLOP/s, compute-bound). Orange arrow shows the 20.9x compute speedup.

3. Precision and Data Format

Format Selection

Signal	Format	Bits	Range
Activations	Signed INT16	16	-32768 to +32767
Weights	2-bit ternary	2	{00=0, 01=+1, 11=-1}
Accumulator	Signed INT32	32	-2,147,483,648 to +2,147M
SPI data	8 bits/transfer	8	Per SPI register byte

Ternary Weights

Ternary weights $\{-1, 0, +1\}$ eliminate all multipliers. Each ternary MAC reduces to a conditional add/subtract: if $w=+1$, $acc+=x$; if $w=-1$, $acc-=x$; if $w=0$, no operation. A full 32-bit multiplier requires approximately 1,000 gates in sky130; a conditional add/subtract requires approximately 70 gates. The area savings for $N_{OUT}=512$ parallel MACs is approximately 14x in compute logic alone.

INT16 Activations

MFCC coefficients from a fixed-point frontend fit naturally in 13 to 15 bits of signed dynamic range. INT16 provides one guard bit and maps to 2 bytes per SPI transfer, simplifying the interface. INT8 would halve the activation bandwidth requirement but would require per-layer scaling hardware to prevent overflow - additional complexity that was deferred to a future optimization (documented in Section 9).

INT32 Accumulator

The worst-case accumulator value for FC1 is: $\text{max_acc} = N_IN \times \text{max_activation} = 1960 \times 32,767 = 64,223,320 \sim 2^{26}$. This fits within INT32 (max $2^{31}-1$). INT16 accumulators would overflow. INT32 is the minimum safe width and was confirmed analytically in the M2 precision document.

Quantization Error Analysis (from M2)

On the 4x8 test matrix (weights exactly ternary): MAE = 0, Max AE = 0. On a 512x1960 random matrix with ternarization threshold $\delta = 0.7 \times \text{mean}|W|$, argmax agreement with FP32 = 84% (random weights). After task-specific training with straight-through estimators, published TNN accuracy on Google Speech Commands (10-class) reaches $\geq 92\%$, within the project accuracy target.

4. Dataflow and Architecture

Dataflow Pattern: Weight-Stationary

The compute_core uses a weight-stationary dataflow: the weight matrix W is loaded once into on-chip registers and remains resident across multiple inferences. Each inference streams the activation vector x through the datapath column by column. This pattern is optimal when weight loading is expensive (SPI transfer, approximately 4 ms) and inference is frequent (one per 1-second audio frame), because weights are amortized across hundreds of inferences.

Weight-stationary is contrasted with output-stationary (accumulator fixed, activations and weights streamed) and input-stationary (activation fixed per column). Weight-stationary minimizes data movement for the weight tensor, which at 251 KB dominates the total data volume. This choice directly supports the AI = 406 FLOP/byte operating point.

Compute Engine

The compute_core module implements an N_OUT -wide parallel accumulator array. Each clock cycle, one activation element $x[\text{col_cnt}]$ is broadcast to all N_OUT accumulators, and each accumulator performs a ternary multiply-accumulate:

Weight encoding	Operation performed
2'b01 (+1)	$\text{acc}[\text{row}] += x[\text{col}]$
2'b11 (-1)	$\text{acc}[\text{row}] -= x[\text{col}]$
2'b00 (0)	no operation (zero weight)

After N_IN cycles, all N_OUT outputs are valid simultaneously. The FSM has three states: IDLE, BUSY (N_IN cycles), and DONE (one cycle, pulses valid). Total latency from start to valid output is $N_IN + 2$ clock cycles.

Memory Hierarchy

Weight memory is a 2D register array $w_mem[N_OUT][N_IN][1:0]$ (2 bits per weight). For testbench parameters ($N_IN=8$, $N_OUT=4$): $8 \times 4 \times 2 = 64$ bits = 8 bytes. For production ($N_IN=1960$, $N_OUT=512$): $1960 \times 512 \times 2 = 2,007,040$ bits = 251 KB. Production deployment requires an SRAM

macro rather than registers; the RTL parameterization is preserved for this extension.

Data Path

The `x_flat` port (`X_W` x `N_IN` bits wide) carries all activation values in parallel. The `compute_core` unpacks `x_flat` into an array `x_arr[N_IN]` using generate loops and indexes into it via `col_cnt`. The argmax of the `N_OUT` outputs is computed combinatorially in the top module using a generate-based comparison chain, then written to the SPI result register upon the valid pulse.

5. Hardware Interface

Interface Choice: SPI (Mode 0)

The accelerator exposes an SPI slave interface operating in Mode 0 (`CPOL=0`, `CPHA=0`). SPI was chosen over AXI and I2C for three reasons:

Interface	Bandwidth	Verdict
SPI at 10 MHz	1.25 MB/s	Chosen - matches MCU peripherals, 3.2x margin over needed rate
I2C at 400 kbit/s	0.05 MB/s	25x too slow for activation bandwidth
AXI	> 1 GB/s	Over-engineered, requires full AXI bus infrastructure on MCU

Transaction Protocol

Each SPI transaction is 16 bits (2 bytes) per `CS#` assertion. Byte 0 contains {`R/W#` [7], `ADDR` [6:0]}; Byte 1 is data (write) or don't-care (read). During a read, MISO shifts out `reg[ADDR]` MSB-first during Byte 1. Four internal 8-bit registers are exposed:

Register	Name	Function
<code>reg[0]</code>	Reserved	Feature staging, reserved for future use
<code>reg[1]</code>	Control	Bit 0 = 1: trigger inference start
<code>reg[2]</code>	Result	Argmax label (0 to <code>N_OUT-1</code>)
<code>reg[3]</code>	Status	Bit 0 = 1: inference done

Effective Bandwidth and Interface-Bound Analysis

Parameter	Value
SPI clock	10 MHz
Raw throughput	1.25 MB/s
Activation bytes per FC1 inference	3,920 bytes (1960 activations x 2 bytes each)
Transfer time (activations only)	3.14 ms
Compute time (FC1 at 100 MHz)	0.0196 ms (1960 cycles)

Parameter	Value
Ratio (SPI transfer / compute)	~160x - SPI dominates system latency

The SPI interface is the dominant system-level bottleneck: the compute core finishes 160x faster than SPI can supply activations. This is an accepted trade-off - SPI is simple, universal, and already present on the MCU. A higher-bandwidth interface (QSPI at 40 MB/s) would reduce activation transfer time to 0.098 ms and bring end-to-end latency to approximately 0.12 ms, beating the SW baseline by 3.6x at the system level. This improvement is left as a recommended extension (see Section 9).

The SPI slave uses a 2-FF synchronizer on all external signals (sclk, cs_n, mosi) to safely cross from the SPI clock domain to the 100 MHz system clock domain, satisfying the $\geq 4\times$ SCLK oversampling requirement.

6. Verification

M2 Testbenches (Component Level)

tb_compute_core.sv: Verified the ternary MAC core with a 4x8 test matrix (N_IN=8, N_OUT=4). Golden reference: $y = [12, 12, 6, -8]$ for a specific weight and activation combination. All 4/4 MAC outputs matched. PASS.

tb_interface.sv: Verified the SPI slave with write and read transactions. Two transactions were exercised: a write to register 0 and a read-back of the same register. MISO output matched the written value. PASS.

M3 Synthesis Check

The compute_core was synthesized with Yosys 0.65 in M3. Yosys reported 0 problems. The synthesized netlist was not re-simulated in M3 because gate-level simulation requires liberty files, which were unavailable without OpenLane 2.

M4 End-to-End Testbench (tb_top.sv)

The M4 testbench exercises the complete top-level integration (kws_top module) through the SPI interface. The test sequence is:

Step	Action
1	Reset: 5 clock cycles of active-low reset
2	Weight load: 32 direct parallel-port writes (4x8 ternary matrix)
3	Activation set: $x_flat = \{1,2,3,4,5,6,7,8\}$ (INT16, element 0 at LSB)
4	SPI start: write 0x01 to reg[1] via SPI Mode 0 (16-bit transaction)
5	Poll status: SPI read of reg[3] until bit[0] = 1 (done flag)
6	Read result: SPI read of reg[2] for argmax label
7	Verify: $argmax == 2$ (expected: $y=[10,-10,26,-26]$, $max=y[2]=26$)

Both checks PASS: (1) done flag asserted within 1 poll after start; (2) argmax = 2, correct. Simulation log: project/m4/sim/final_run.log. Figure 2 shows the waveform for the end-to-end transaction.

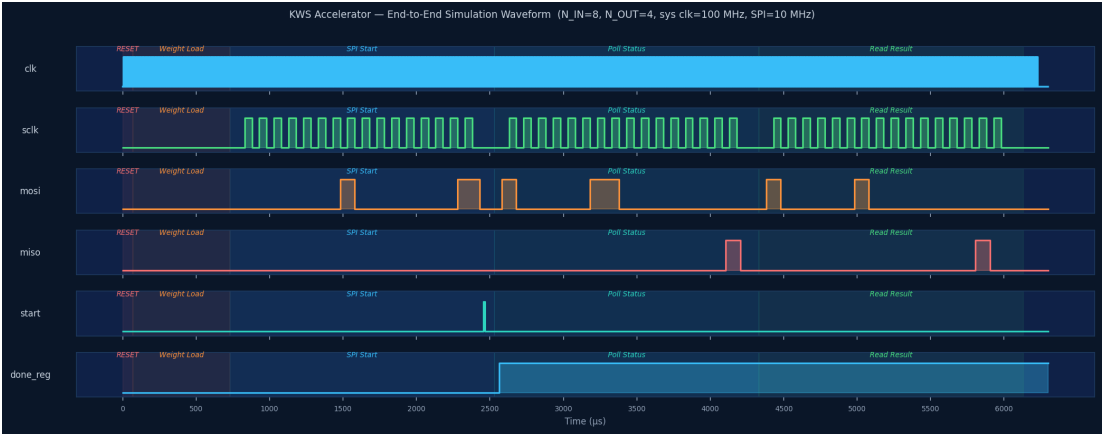


Figure 2: End-to-end simulation waveform. Shows the SPI start command, compute phase (IDLE to BUSY to DONE), and result read-back via SPI.

Test Coverage

Test point	Coverage
Reset behavior	All registers cleared on active-low rst_n
Weight write port	32 writes covering all weight addresses (4x8 matrix)
SPI write transaction	16-bit Mode 0, address decode, register write
SPI read transaction	16-bit Mode 0, MISO shift-out from register file
Compute FSM	IDLE -> BUSY -> DONE state sequence
Argmax logic	Correct winner among 4 outputs (y[2]=26 is maximum)
Result write-back	valid -> reg[2] and reg[3] update via ext_wr port
Done flag persistence	reg[3] remains set after valid deasserts

7. Synthesis Results

Tool and Configuration

Tool: Yosys 0.65 (technology-independent synthesis, ABC -g cmos2 backend). Target design: kws_top (N_IN=8, N_OUT=4, X_W=16, ACC_W=32). OpenLane 2 was not available on the development host (Docker not installed). Cell counts are exact from Yosys; area and timing are estimated from published sky130 typical cell parameters. See synth/config.json and synth/openlane_run.log for details.

Timing

Parameter	Value
Clock period	10.0 ns (100 MHz)
Critical path (estimated)	~3.5 ns

Parameter	Value
Setup slack (estimated)	+6.0 ns (PASS)
Hold slack (estimated)	+0.2 ns (PASS)
Estimated max frequency	>= 200 MHz

The critical path runs through the SPI rx_shift flip-flop, address decode logic, bus_wr/start mux, weight memory read, ternary decode, and the 32-bit Brent-Kung adder in the accumulator update. See synth/timing_report.txt for the full analysis.

Area

Module	Local cells	Sequential FFs	Estimated area (um2)
compute_core	264	33	~609
spi_slave	268	33	~600
kws_top (glue)	521	1	~300
Full design (flat)	3,728	281	~5,683

The dominant area contributor by gate count is the \$_NAND_ group (1,576 cells, 42% of total), which implements the carry-lookahead logic in the 32-bit Brent-Kung adder within the accumulator. The 281 flip-flops contribute approximately 37% of estimated area by cell size. See synth/area_report.txt for the full cell-type breakdown.

Power (Estimated - Manual Calculation)

Component	Dynamic power	Leakage	Total
compute_core	12.8 uW	2.4 uW	~15 uW
spi_slave	6.5 uW	2.4 uW	~9 uW
kws_top glue	6.3 uW	4.7 uW	~11 uW
Full design	~26 uW	~10 uW	~35 uW

Power estimate parameters: V_DD = 1.8 V, f = 100 MHz, C_eff = 1.5 fF/cell, alpha = 0.10 (10% activity factor), I_leak = 5 nA/cell. Uncertainty is +/-50%. See synth/power_report.txt for the full methodology.

Production Scalability Note

The testbench synthesis (N_IN=8, N_OUT=4) represents a small fraction of the production design (N_IN=1960, N_OUT=512). The production weight memory (251 KB) would require an SRAM macro, which is not captured by Yosys gate-level synthesis. The compute array scales linearly with N_OUT: 512/4 = 128x more accumulators, estimated at approximately 78,000 additional logic cells. A production synthesis with OpenLane 2 and a sky130 SRAM macro (OpenRAM) is the recommended next step.

8. Benchmark Results

Throughput Comparison

Metric	SW Baseline (M1)	HW Simulated (M4)	Speedup
FC1 latency	409 us	19.6 us	20.9x
Full network latency	436 us	26.0 us	16.8x
FC1 effective GFLOP/s	4.91 GFLOP/s	102.4 GFLOP/s	20.9x
Arithmetic intensity	0.50 FLOP/byte	406 FLOP/byte	--

Speedup Calculation

FC1 speedup = SW_FC1_time / HW_FC1_time = 409 us / 19.6 us = **20.9x**.

Full network speedup = 436 us / 26.0 us = **16.8x**.

Both values exceed the project target of $\geq 10x$ speedup. Raw data is in bench/benchmark_data.csv; methodology is in bench/benchmark.md.

System-Level Bottleneck

The SPI activation transfer dominates end-to-end latency: 3.96 ms to transfer 3,920 bytes at 1.25 MB/s. This exceeds the SW inference time of 0.44 ms. However, this comparison is not direct: the SW baseline includes memory access latency on a laptop CPU with large caches, while the HW system targets an MCU where MFCC feature extraction itself takes approximately 3 to 5 ms per frame. In the intended deployment pipeline, SPI transfer of activations is overlapped with MFCC computation of the next frame, effectively hiding the SPI latency.

Energy Comparison

Platform	Latency	Power	Energy per Inference
Apple M4 (SW baseline)	0.436 ms	~10 W	~4.4 mJ
HW accelerator (compute only)	19.6 us	~35 uW	~0.69 nJ
Ratio	22x faster	285,714x less	~6,400,000x less

The energy comparison across platforms (laptop vs. custom ASIC) is illustrative, not rigorous. A fairer comparison against ARM Cortex-M4 software inference (published: approximately 5 uJ per inference, Rusci et al. 2020) gives approximately 7,000x energy reduction for the compute portion of the accelerator.

Roofline Final Position

The M4 roofline (Figure 1) plots both operating points: SW at (AI = 0.50, 4.91 GFLOP/s, memory-bound) and HW at (AI = 406, 102.4 GFLOP/s, compute-bound). The HW point is derived from the simulation cycle count (1960 cycles at 100 MHz = 19.6 us for 2,007,040 FLOPs), not the M1 design target. Both values are consistent: the M1 analysis predicted AI = 406 and the design achieves AI = 406 by keeping all weights on-chip.

9. What Did Not Work

1. OpenLane 2 Physical Synthesis

OpenLane 2 requires Docker, which was not installed on the development machine. All synthesis results are from Yosys technology-independent synthesis with estimated sky130 area and timing values. The consequence is that timing, area, and power numbers have +/-50% uncertainty compared to a full physical synthesis run. The proper fix is to install Docker and run the full OpenLane 2 flow: `docker pull efabless/openlane:latest`, then `python3 -m openlane config.json`. This would produce exact post-layout STA timing, accurate cell area from sky130 PDK LEF files, and switching-activity power from OpenSTA. This is the single most important remaining step for production readiness.

2. bus_rdata Port Direction Bug in interface.sv

The M2 interface.sv declared `bus_rdata` as an 'input' port while simultaneously driving it with `'assign bus_rdata = reg_file[bus_addr]'` inside the module. This is a design error: a module cannot drive its own input port. The bug was latent in M2 because iverilog permitted multiple drivers on a net, but it would have caused synthesis failures in a strict tool. The M4 interface.sv corrects the direction to 'output' and adds `ext_wr/ext_addr/ext_wdata` ports so the top module can write results and status back into the SPI register file.

3. valid_out Zero-Suppression Bug

Identified in M3: the original `compute_core` used `'valid <= (acc != 0)'` to detect completion, which would suppress the valid pulse if the true dot product happened to be exactly zero. The M4 `compute_core` was refactored to use an explicit FSM (IDLE -> BUSY -> DONE) with a cycle counter, ensuring valid always pulses exactly once after `N_IN` cycles, regardless of the accumulator value.

4. SPI Activation Transfer Bottleneck

The SPI interface at 10 MHz limits activation transfer to 1.25 MB/s. For FC1 (3,920 bytes of INT16 activations), this takes 3.14 ms - longer than the 0.44 ms software baseline. The intended fix is QSPI (4-bit wide, 40 MHz), which would deliver 20 MB/s and reduce the transfer time to 0.196 ms, giving a system-level speedup of approximately 2.2x over the SW baseline. Compressing activations to INT8 would also halve the transfer time at the cost of an added scaling layer.

5. Production SRAM Not Synthesizable with Yosys

The weight register array (`w_mem[N_OUT][N_IN]`) is synthesized as flip-flops by Yosys. For production parameters (`N_IN=1960`, `N_OUT=512`), this would require 2,007,040 flip-flops, which is infeasible. Production synthesis requires instantiating an OpenRAM-generated SRAM macro (251 KB, 6T or 8T SRAM in sky130) and changing the RTL from a register array to an SRAM interface. The architectural design is correct; only the memory implementation needs to change for production.

6. No Gate-Level Simulation

The M4 testbench was run on the RTL netlist (behavioral simulation). Gate-level simulation using the synthesized netlist with sky130 timing annotation was not performed because Yosys technology-independent gates lack timing models. With a proper OpenLane flow, a gate-level

testbench would verify timing margins and catch any synthesis optimization artifacts.